TOPIC 8: Functions

TASK 1: Guided Activity, Marks and grades toolkit

```
func average(of marks: Int...) -> Double {
    guard !marks.isEmpty else {
        return 0
    }
    let total = marks.reduce(0, +)
    return Double(total) / Double(marks.count)
}
func grade(for average: Double) -> String {
    switch average {
    case 90...:
        return "O"
    case 80..<90:
        return "A+"
    case 70..<80:
        return "A"
    case 60..<70:
        return "B"
    default:
        return "RA"
    }}
func stats(of marks: [Int]) -> (min: Int, max: Int, average: Double)? {
    guard !marks.isEmpty else {
        return nil
    }
    let minimum = marks.min()!
    let maximum = marks.max()!
    let avg = Double(marks.reduce(0, +)) / Double(marks.count)
```

TOPIC 8: Functions

```
    return (minimum, maximum, avg)
}
func curve(_ marks: inout [Int], by bonus: Int = 5) {
    for i in marks.indices {
        marks[i] = min(marks[i] + bonus, 100)
    }}
print("Average:", average(of: 85, 90, 95))
print("Empty Average:", average(of:))
print("Grade:", grade(for: 85))
var marks = [65, 78, 90, 98
if let result = stats(of: marks) {
    print("Minimum:", result.min)
    print("Maximum:", result.max)
    print("Average:", result.average)
} else {
    print("No marks available")
}
curve(&marks)
print("Marks after bonus:", marks)
```

OUTPUT:

```
Output:
Average: 90.0
Empty Average: (Function)
Grade: A+
Minimum: 65
Maximum: 98
Average: 82.75
Marks after bonus: [70, 83, 95, 100]
```

TOPIC 8: Functions

TASK 2: Practice Task, Text utilities

```
import Foundation


func isPalindrome(_ text: String) -> Bool {
    let cleaned = text.lowercased()
        .filter { $0.isLetter || $0.isNumber }
    return cleaned == String(cleaned.reversed())
}
func vowelCount(in text: String) -> Int {
    let vowels = "aeiou"
    return text.lowercased()
        .filter { vowels.contains($0) }
        .count
}
func wordCount(in text: String) -> Int {
    let words = text.split { $0.isWhitespace }
    return words.count
}
func wordCount(in text: String, minLength: Int) -> Int {
    let words = text.split { $0.isWhitespace }
    return words.filter { word in
        let letters = word.filter { $0.isLetter }
        return letters.count >= minLength
    }.count
}
let sentence = "A man, a plan, a canal: Panama"
print("Sentence:", sentence)
print("Is Palindrome:", isPalindrome(sentence))
```

TOPIC 8: Functions

```
print("Vowel Count:", vowelCount(in: sentence))
print("Word Count:", wordCount(in: sentence))
print("Words with at least 4 letters:",
    wordCount(in: sentence, minLength: 4))
```

OUTPUT:

```
Output:
Sentence: A man, a plan, a canal: Panama
Is Palindrome: true
Vowel Count: 10
Word Count: 7
Words with at least 4 letters: 3
```

TASK 3: Challenge Task, Functions as values

```
func makeMultiplier(_ factor: Int) -> (Int) -> Int {
   return { number in
      number * factor
   }
}
let double = makeMultiplier(2)
let triple = makeMultiplier(3)
func apply(_ values: [Int], using transform: (Int) -> Int) -> [Int] {
   return values.map(transform)
}
let numbers = [1, 2, 3, 4, 5]
print("Original:", numbers)
print("Doubled:", apply(numbers, using: double))
```

TOPIC 8: Functions

```
print("Tripled:", apply(numbers, using: triple))

let squared = apply(numbers) { number in

   number * number

}

print("Squared:", squared)
```

OUTPUT:

Output:

```
Original: [1, 2, 3, 4, 5]
Doubled: [2, 4, 6, 8, 10]
Tripled: [3, 6, 9, 12, 15]
Squared: [1, 4, 9, 16, 25]
```